



TP 5 : Régression linéaire simple et multiple avec Python

Objectifs du TP

- Comprendre le principe de la régression linéaire.
- Implémenter une régression linéaire en Python :
 - Avec **numpy** (méthode analytique).
 - Avec **scikit-learn** (modèle prêt à l'emploi).
- Interpréter les coefficients, l'erreur et le score.
- Visualiser les résultats (droite de régression, résidus).
- Étendre vers la **régression multiple**.

Partie 1 : Régression linéaire simple

1. Chargement des données

On utilisera un dataset simple : taille → poids.

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Jeu de données (exemple)
```

```
data = {
```

```
    "taille": [150, 160, 170, 180, 190],
```

```
    "poids": [50, 60, 65, 75, 85]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
X = df["taille"].values.reshape(-1, 1)
```

```
y = df["poids"].values
```

Questions

1. Afficher les 5 premières lignes du DataFrame.
2. Tracer le nuage de points `taille` vs `poids`.

2. Régression linéaire avec scikit-learn

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X, y)
```

```
print("Coefficient (pente) :", model.coef_[0])
```

```
print("Ordonnée à l'origine :", model.intercept_)
```

```
print("Score R2 :", model.score(X, y))
```

Questions

1. Interpréter la pente.
2. Que représente le score R² ?
3. Prédire le poids pour une taille de 175 cm.

3. Visualisation de la droite de régression

```
plt.scatter(X, y, color='blue')
```

```
plt.plot(X, model.predict(X), color='red')
```

```
plt.xlabel("Taille (cm)")
```

```
plt.ylabel("Poids (kg)")
```

```
plt.title("Régression linéaire")
```

```
plt.show()
```

Questions

1. La droite ajuste-t-elle bien les données ? Pourquoi ?

Partie 2 : Régression linéaire par méthode analytique (formule matricielle)

Rappel :

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

4. Implémentation

```
# Ajout du biais (colonne de 1)
X_b = np.c_[np.ones((len(X), 1)), X]

# Calcul analytique
theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
print("Theta (intercept, slope) :", theta)
```

Questions

1. Comparer θ obtenu par numpy avec les coefficients du modèle sklearn.
2. Pourquoi ajoute-t-on une colonne de 1 dans X ?

Partie 3 : Analyse des résidus

5. Calcul des résidus

```
y_pred = model.predict(X)
residus = y - y_pred
plt.scatter(X, residus)
plt.axhline(0, color='red')
plt.xlabel("Taille")
plt.ylabel("Résidus")
plt.title("Analyse des résidus")
plt.show()
```

Questions

1. Les résidus semblent-ils aléatoires ?
2. Que signifie la présence d'un motif particulier ?

Partie 4 : Régression linéaire multiple

6. Jeu de données élargi

```
data = {  
    "taille": [150, 160, 170, 180, 190],  
    "age": [20, 25, 30, 35, 40],  
    "poids": [50, 60, 65, 75, 85]  
}  
  
df = pd.DataFrame(data)  
X = df[["taille", "age"]]  
y = df["poids"]
```

7. Modèle multiple

```
model = LinearRegression()  
model.fit(X, y)  
print("Coefficients :", model.coef_)  
print("Intercept :", model.intercept_)  
print("Score R2 :", model.score(X, y))
```

Questions

1. Interpréter les deux coefficients.
2. Comment savoir si l'âge est pertinent dans la prédiction ?

Partie 5 : Validation croisée

```
from sklearn.model_selection import cross_val_score  
  
scores = cross_val_score(LinearRegression(), X, y, cv=5)  
  
print("Scores :", scores)  
print("Score moyen :", scores.mean())
```

Questions

1. Pourquoi faire une validation croisée ?
2. Que signifie un score qui varie beaucoup selon les folds ?

Questions supplémentaires

Niveau 1 – Compréhension de base

1. **Que se passe-t-il si on change l'échelle des variables ?**
(Ex : taille en mètres vs centimètres)
2. **Que représente exactement l'ordonnée à l'origine ?**
Est-elle toujours interprétable dans le contexte ?
3. **Pourquoi est-il important de visualiser les données avant d'appliquer un modèle ?**
4. **Quelles hypothèses fondamentales doit respecter un modèle de régression linéaire ?**

Niveau 2 – Analyse technique

5. **Tracer la distribution des résidus** (histogramme).
→ Ont-ils une forme proche d'une distribution normale ?
6. **Calculer l'erreur quadratique moyenne (MSE) :**

```
from sklearn.metrics import mean_squared_error
```



```
mse = mean_squared_error(y, y_pred)
```

 - Comment interpréter le MSE ?
 - Est-il cohérent avec le graphique des résidus ?
7. **Comparer la régression linéaire à un modèle naïf** (ex : prédire la moyenne).
 - Dans quels cas le modèle linéaire est-il réellement meilleur ?
8. **Tester le modèle sur un point très éloigné du dataset** (ex : taille = 210 cm).
 - Que constates-tu ?
 - Qu'est-ce que cela dit sur l'extrapolation ?

Niveau 3 – Régression multiple

9. **Calculer la matrice de corrélation** des variables :

```
df.corr()
```

 - Que peut-on dire de la corrélation taille-âge-poids ?
 - Une variable semble-t-elle redondante ?
10. **Tester un modèle sans la variable "âge" :**
 - Le R^2 augmente-t-il ou diminue-t-il ?
 - Comment l'expliquer ?
11. **Vérifier la colinéarité des variables** (Variance Inflation Factor – VIF).
→ Quelle variable semble la plus problématique ?